CVM++

A Custom Interpreted Programming Language in C++

Project Report

Submitted by: Kushal Lakshya Tiwari
Roll No.: 240101051
Department: CSE
Institution: IIT Guwahati

Date: May 20, 2026

Submitted in partial fulfillment of the requirements for [Course/Degree]

Abstract

CVM++ is a fully functional, interpreted programming language implemented in C++. The project demonstrates a complete language execution pipeline: a hand-written lexer and tokenizer, a recursive descent parser that constructs an Abstract Syntax Tree (AST), a bytecode compiler that translates the AST into intermediate instructions, and a stack-based virtual machine that executes those instructions. This report describes the design, architecture, implementation, and testing of each component.

Contents

1 Introduction

Programming languages are the foundation of all software systems. While high-level languages abstract away low-level execution details, understanding how source code is transformed into executable instructions is essential for any serious computer science student.

CVM++ was developed to bridge this gap. By building each stage of a language interpreter from scratch in C++, this project provides a practical exploration of the following concepts:

- Lexical analysis and tokenization
- Syntax analysis via recursive descent parsing
- Intermediate representation using bytecode
- Execution through a stack-based virtual machine

CVM++ is not merely a theoretical exercise — it is a working language implementation capable of parsing and executing programs written in its custom `.cvm` syntax.

2 Objectives

The primary objectives of this project are:

1. Design and implement a working programming language from scratch in C++.
2. Build a hand-written lexer and tokenizer to process raw source text.
3. Implement a recursive descent parser to produce an Abstract Syntax Tree (AST).
4. Develop a bytecode compiler that translates the AST into executable instructions.
5. Create a stack-based virtual machine to execute the generated bytecode.
6. Ensure the system is modular, readable, and extensible.

3 System Design and Architecture

CVM++ is structured as a sequential pipeline. Source code passes through each stage before final execution:

`Source Code (.cvm)` → `Lexer / Tokenizer` → `Parser (AST)` → `Bytecode Compiler` → `Stack-Based VM`

Figure 1: CVM++ Execution Pipeline

3.1 Lexer and Tokenizer

The lexer reads the source file character by character and groups characters into tokens. Each token carries a type (defined in `Tokens.h`) and its literal value. The tokenizer handles whitespace, comments, literals, identifiers, keywords, operators etc.

3.2 Recursive Descent Parser

The parser (implemented in `Parser.h` / `parser.cpp`) consumes the token stream and applies grammar rules using mutually recursive functions. The output is an Abstract Syntax Tree (AST), whose node types are defined in `ASTNodes.h`. This approach naturally encodes operator precedence through the function call hierarchy.

The current Grammar used for this project:
program → statement* EOF ;
statement → IfStmt |WhileStmt |PrintStmt |InputStmt |ExprStmt ;
IfStmt → ”if” ”(”expression”)” statement ”;” ;
WhileStmt — ”while” ”(”expression”)” statement ”;” ;
InputStmt → ”input” expression ”;” ;
PrintStmt → ”print” expression ”;” ;
ExprStmt → expression ”;” ;

expression → equality ;
assignment → identifier ”=” assignment ;
equality → comparison ((”==” comparison)* ;
comparison → term ((”<” |”>”) term)* ;
term → factor ((”-” |”+”) factor)* ;
factor → unary ((”/” |”*”) unary)* ;
unary → (”!”) unary |primary ;
primary → number |boolean |identifier |”(” expression ”)” ;

3.3 Bytecode Compiler

The bytecode compiler performs a tree-walk over the AST and emits a sequence of instructions into a `Chunk` (defined in `chunk.h`) using the **Visitor Pattern**. A `ValueArray` (`ValueArray.h`) stores the constant pool. Jump targets for control flow are resolved via backpatching after the target bytecode is emitted.

3.4 Stack-Based Virtual Machine

The virtual machine (`VirtualMachine.h`) maintains an operand stack, an instruction pointer, hash table for storing global variables. It fetches each opcode, decodes it, and executes the corresponding operation. Variable storage is backed by a `HashTable` (`HashTable.h`).

Table 1: CVM++ Pipeline Stages and Key Files

Stage	Component	Key Files
1	Lexer / Tokenizer	`Lexer.h`, `Lexer.cpp`, `Tokens.h`
2	Recursive Descent Parser	`Parser.h`, `parser.cpp`, `ASTNodes.h`
3	Bytecode Compiler	`chunk.h`, `ValueArray.h`, `Compiler.h`
4	Stack-Based Virtual Machine	`VirtualMachine.h`, `HashTable.h`, `memory.h`

4 Implementation

4.1 Lexer (Lexer.h / Lexer.cpp)

The lexer scans source text character by character, identifies token boundaries, and produces a flat list of tokens. Each token carries its type enum class (`TokenType` from `Tokens.h`), its raw lexeme, and its line number for error reporting.

```
struct Token{
    TokenType type = TokenType::Default;
    std::string val;
    int line;
    Token(TokenType type, int line) : type(type), val(""), line
       (line){}
    Token(TokenType type, std::string val, int line) : type(
       type), val(val), line(line){}
};
```

Listing 1: Token structure (Tokens.h)

4.2 Parser (Parser.h / parser.cpp)

The recursive descent parser converts the token stream into an AST. Each grammar rule corresponds to a C++ function. These functions call each other recursively, naturally encoding precedence. Node types (expression, statement, declaration, etc.) are defined in `ASTNodes.h`.

```
std::unique_ptr<ExpressionAST> Parser::term()
{
    auto expr = factor();
    while (match({TokenType::Plus, TokenType::Minus}))
    {
        Token op = previous();
        expr = std::make_unique<Binary>(op, std::move(expr),
           factor());
    }
    return expr;
}
std::unique_ptr<ExpressionAST> Parser::factor()
{
    auto expr = unary();
    while (match({TokenType::Mul, TokenType::Div}))
    {
        Token op = previous();
        auto right = unary();
        expr = std::make_unique<Binary>(op, std::move(expr),
           std::move(right));
    }
    return expr;
}
```

Listing 2: Example recursive descent function skeleton

4.3 Bytecode Compiler (chunk.h / ValueArray.h)

The compiler performs a post-order traversal of the AST and emits opcodes into a `Chunk`. Constants (numbers, strings) are stored in a `ValueArray`. Control flow (if/else, while) uses backpatching to resolve jump targets.

```
class Chunk{
public:
    int capacity;
    int count;
    int *lines;
    uint8_t *code;
    ValueArray constants;
    void initChunk(){
```

Listing 3: Chunk structure (chunk.h)

4.4 Virtual Machine (VirtualMachine.h)

The VM maintains an operand stack and an instruction pointer. It fetches each opcode in a loop and dispatches to the appropriate handler. Local and global variables are stored in a `HashTable`.

```
InterpretResult run()
    {
        // READ_BYTE, READ_CONSTANT, BINARY_OP are defined
           first.

        // Main execution below
        for (;;)
        {
            Opcode instruction;
            switch (instruction = (Opcode)READ_BYTE())
            {
            case (Opcode::Op_Return):
            {
                return InterpretResult::INTERPRET_OK;
                break;
            }
            case Opcode::Op_Constant:
            {
                Value rconstant = READ_CONSTANT();
                push(rconstant);
                break;
            }
```

Listing 4: VM execution loop skeleton

4.5 Supporting Modules

- **`HashTable.h`** — Open-addressing hash map for global variable storage.
- **`memory.h`** — Centralized allocation and dynamic array growth utilities.

- **commons.h** — Common type aliases.
- **error.h** — Error reporting with line number and descriptive messages. (Not completed yet)

5 Bytecode Instruction Set

Table 2: CVM++ Opcode Reference

Opcode	Description
OP_CONSTANT	Push a constant from the pool onto the stack
OP_ADD	Pop two values, push their sum
OP_SUB	Pop two values, push their difference
OP_MUL	Pop two values, push their product
OP_DIV	Pop two values, push their quotient
OP_EQUALITY	Push true if top two values are equal
OP_LESSER	Push true if second value < top value
OP_GREATER	Push true if second value > top value
OP_NOT	Logical NOT of the top value
OP_JUMP_IF_FALSE	Jump if top of stack is false/zero
OP_JUMP	Jump backwards (for while loops)
OP_GET_GLOBAL	Push value of a global variable
OP_SET_GLOBAL	Assign value to a global variable
OP_PRINT	Pop and print the top value
OP_INPUT	Take input directly from user and push it onto the stack
OP_RETURN	End VM execution

6 Testing and Results

The following test cases were used to validate the CVM++ pipeline end-to-end:

Table 3: Test Cases and Results

Test Case	Description	Result
Hello World	Print a string literal	Pass
Arithmetic	Basic +, -, *, / operations	Pass
Variables	Variable declaration and assignment	Pass
Conditionals	if branching	Pass
Loops	while loop with counter	Pass

7 Challenges and Solutions

7.1 Operator Precedence in the Parser

Challenge: Correctly handling operator precedence without explicit precedence tables.
Solution: Modeled precedence through the grammar hierarchy in the recursive descent

parser. Lower-precedence rules call higher-precedence ones, naturally producing correct evaluation order.

7.2 Jump Backpatching in the Compiler

Challenge: When compiling `if/else` and `while` constructs, jump target offsets are unknown at emit time.
Solution: Emitted placeholder jump offsets and backpatched them after generating the target bytecode.

7.3 Memory Management

Challenge: Managing dynamic memory safely in C++ without leaks or dangling pointers.
Solution: Centralized allocation through `memory.h` using `GROW_ARRAY` and `FREE_ARRAY` macros for consistent, traceable allocations.

7.4 Emitting bytecode

Challenge: Clean, Readable and Manageable code for bytecode generation.
Solution: Use a **Visitor pattern** which makes a common accept function for all nodes. All the emit functions are inside Compiler.h which can be easily edited and managed.

8 Conclusion

CVM++ successfully demonstrates a complete language implementation pipeline in C++. Starting from raw source text, the system performs lexical analysis, recursive descent parsing, bytecode compilation, and stack-based execution in a clean, modular architecture.

Building CVM++ provided deep practical insight into how programming languages work at each stage of execution — from character streams and token recognition, through syntax analysis and tree construction, to bytecode generation and runtime execution. The project serves as both a functional interpreter and an educational reference for language implementation in C++.

9 Future Work

- Add support for all other unary and binary operators
- Add support for more data types: string, arrays, dictionaries, and user-defined structs.
- Implement a mark-and-sweep garbage collector for automatic memory management.
- Add functions, blocks and local variables
- Develop a REPL (Read-Eval-Print Loop) for interactive use.
- Add syntax error detection in the parser and runtime error detection in the VM for better diagnostics.

References

1. Nystrom, R. (2021). *Crafting Interpreters*. Available at: https://craftinginterpreters.com

A Project File Structure

```
CVM++/
|-- include/
|   |-- ASTNodes.h          AST node type definitions
|   |-- chunk.h             Bytecode chunk (instructions + constant pool)
|   |-- commons.h           Shared macros and type aliases
|   |-- error.h             Error handling and reporting
|   |-- HashTable.h         Hash map for variable storage
|   |-- Lexer.h             Lexer class interface
|   |-- memory.h            Memory allocation utilities
|   |-- Parser.h            Parser class interface
|   |-- Tokens.h            Token type enum and struct
|   |-- ValueArray.h        Dynamic array for VM values
|   |-- VirtualMachine.h    Stack-based VM interface
|   |-- Compiler.h          The compiler which generates bytecode. Visitor pattern is
|   |-- Printer.h           Printer for the AST. Visitor pattern is used.
|
|-- src/
|   |-- Lexer.cpp           Lexer / tokenizer implementation
|   |-- parser.cpp          Recursive descent parser
|   |-- main.cpp            Entry point
|
|-- build/                  Compiled .o object files
|-- programs/
|   |-- program_code.cvm    Sample CVM++ program
|
|-- makefile                Build system
|-- main.exe                Compiled executable
|-- README.txt              Project overview
|-- instructions.txt        Build and usage guide
```

B Sample CVM++ Program

```
print "Hello from CVM !";

x = 10;
y = 20;
print (x + y);
```

Listing 5: programs/program_code.cvm

```
x = 10;
while(x > 5) x = x-2;
print x;
```

Listing 6: programs/program_code.cvm

```
x = input "Enter Number : ";

```

```
if(x > 10) print "x is more than 10 \n";
if(x < 10) print "x is less than 10 \n";
if (x == 10) print "x is equal to 10 \n"

print "Programme Complete.";
```

Listing 7: programs/program_code.cvm

C Build System (Makefile)

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -g -I include

SRC_DIR = src
BUILD_DIR = build

SRCS = $(wildcard $(SRC_DIR)/*.cpp)
OBJS = $(patsubst $(SRC_DIR)/%.cpp, $(BUILD_DIR)/%.o, $(SRCS))
TARGET = main

all: $(BUILD_DIR) $(TARGET)

$(BUILD_DIR):
    mkdir -p $(BUILD_DIR)

$(TARGET): $(OBJS)
    $(CXX) $(CXXFLAGS) -o $@ $^

$(BUILD_DIR)/%.o: $(SRC_DIR)/%.cpp
    $(CXX) $(CXXFLAGS) -c $< -o $@

run: all
    ./$(TARGET)

clean:
    rm -rf $(BUILD_DIR) $(TARGET)

.PHONY: all clean run
```

Listing 8: makefile